



Python Fundamentos

Introducción

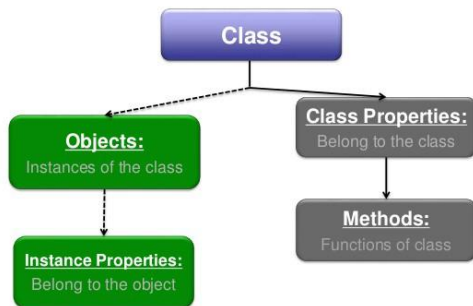
1



Es posible definir Python como un lenguaje de programación multiparadigma. Este término hace referencia a la versatilidad que ofrece a los programadores para adoptar uno o varios estilos de programación, como lo serían la programación funcional o la programación orientada a objetos. En este apartado nos vamos a centrar en desarrollar la idea de clase y objeto.

Una clase se puede definir como una estructura (un molde) diseñada para organizar y encapsular atributos (variables) y métodos (funciones) que están relacionados. La clase sirve como plantilla para crear objetos, también denominados instancias de dicha clase. Cada instancia pues tendrá sus propios atributos y puede ejecutar sus propios métodos.

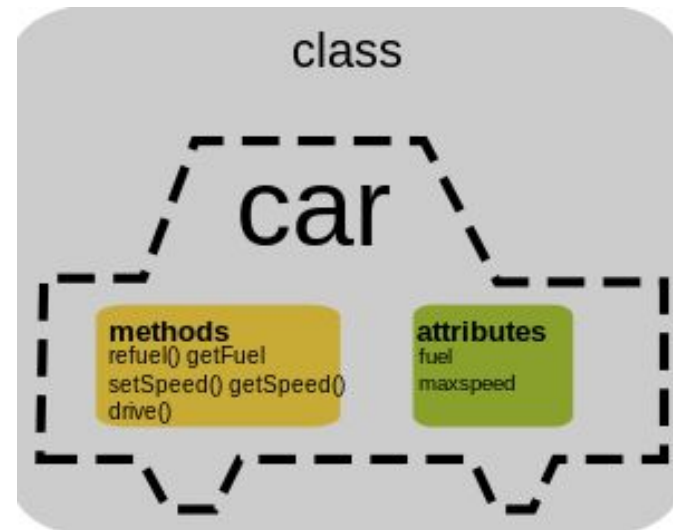
Classes (*objects*)





Hay ciertas características que nos pueden ayudar a entender mejor qué es una clase:

- Las clases tienen como finalidad representar entidades o conceptos
- Cada clase define una serie de métodos adecuados (comportamiento) para operar con un conjunto de variables (estado).
- Las clases sirven como plantilla para crear instancias de datos.
- Las variables no son estáticas, sino que pueden ser modificadas durante la ejecución.
- Tanto los métodos como los atributos son públicos, lo que implica que la responsabilidad recae sobre el buen hacer del programador. Los métodos son virtuales, lo que capacita a las clases derivadas (subclases) de otra clase a sobrescribir dichos métodos heredados.



Clases y objetos

2



La programación orientada a objetos tiene tres características principales:

- Polimorfismo. Se define como una única interfaz, con múltiples métodos con comportamiento distinto dependiendo de la instancia de la interfaz que lo esté invocando.
- Herencia. Es un proceso por el cual un objeto o instancia hereda los métodos y las propiedades de otro, evitando así la necesidad de redefinir estos desde cero.
- Encapsulación. Es un procedimiento mediante el cual las funciones y los datos están contenidos en un objeto para poder acceder a ellos de forma controlada.

Los objetos los podemos definir como entidades que poseen estado, comportamiento e identidad. Podemos pensar en la clase como un molde y el objeto como una pieza sacada de ese molde:

- Para un objeto su identificador es una propiedad que lo diferencia del resto y lo acompaña durante todo su ciclo de vida.
- En Python, los atributos y métodos de una clase están estrechamente relacionados. Los atributos representan el estado de un objeto (las propiedades o datos que mantiene), mientras que los métodos representan el comportamiento (las funciones que operan sobre esos datos).

https://youtu.be/DlphYPc_HKk



La sintaxis básica para crear una clase es la siguiente:

- **‘NombreDeClase’**: Se constituye con el estilo de escritura CamelCase, al contrario por ejemplo que las funciones que utilizan el estilo snake_case.
- **‘variable_de_clase’**: es un atributo de clase compartido por todas las instancias
- **‘__init__’**: se trata de un método especial llamado constructor, que se ejecuta automáticamente cuando se crea una nueva instancia de la clase.
- **‘self’**: es una referencia a la instancia actual de la clase, y se utiliza para acceder a los atributos y métodos de esa instancia.
- **‘método_de_clase’**: realiza operaciones específicas de la clase.

```
python

class NombreDeClase:
    """
    Description
    """

    # Atributos de la clase
    variable_de_clase = "valor"

    def __init__(self, parametro1, parametro2):
        """
        Description
        """
        self.parametro1 = parametro1
        self.parametro2 = parametro2

    def metodo_de_clase(self):
        """
        Description
        """
        return self.parametro1 + self.parametro2
```

Métodos

3



Constructor

El método '___init___' es el encargado de ejecutarse cuando se crea una nueva instancia de una clase dada. Permite realizar operaciones con el objeto y asignar atributos en el mismo momento en el que se crea. Se denomina como constructor. Todas las clases tienen una función llamada '___init___()'.

```
def __init__(self, parametro1, parametro2):  
    """  
    Description  
    """  
    self.parametro1 = parametro1  
    self.parametro2 = parametro2
```

¿Cómo crear un método?

En los objetos los métodos son funciones que pertenecen al mismo y que le habilitan para llevar a cabo operaciones. Se definen exactamente igual que las funciones con 'def'. Hay que tener en cuenta que el parámetro 'self' es una referencia a la instancia actual de la clase, y nos sirve para acceder a las variables de la clase.

```
def metodo_de_clase(self):  
    """  
    Description  
    """  
    return self.parametro1 + self.parametro2
```

Métodos estáticos

4



Se trata de un tipo especial de método que pertenece a una clase en lugar de una instancia específica de esta clase. Los métodos estáticos no requieren una referencia a ninguna instancia y pueden ser invocados directamente desde la clase. Se definen mediante el decorador '@staticmethod' o utilizando la palabra clave 'staticmethod' antes de la definición del método.

```
python

class MiClase:
    @staticmethod
    def metodo_estatico():
        # Código del método estático
        pass
```

```
python

class MiClase:
    def metodo_estatico():
        # Código del método estático
        pass

    metodo_estatico = staticmethod(metodo_estatico)
```

Algunas características clave de los métodos estáticos son:

- Puede ser invocado directamente desde la clase.
- Los métodos estáticos pueden acceder a atributos de clase, pero no pueden acceder a atributos de instancia.
- Los métodos estáticos se utilizan comúnmente para implementar funciones relacionadas con la clase que no dependen del estado de una instancia en particular.

Comparaciones y operaciones

5



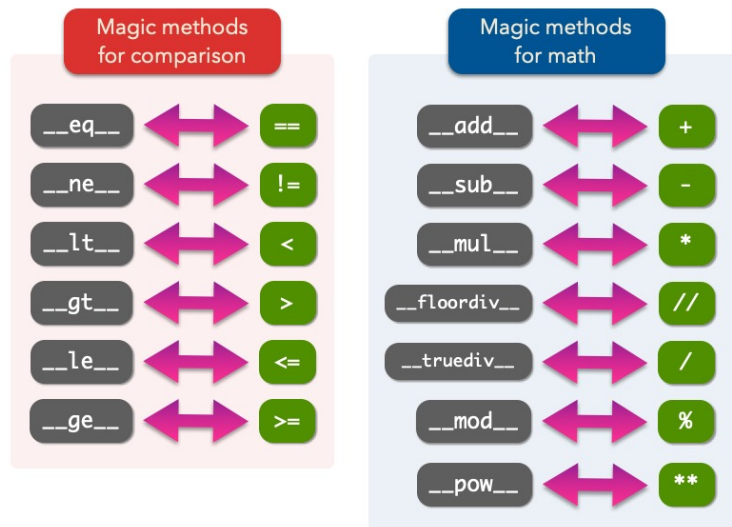
Para Python es posible realizar comparaciones entre objetos, pero se necesita la declaración de un método específico dentro de la clase para ello. En la imagen inferior se puede apreciar que el método '___eq___' ha sido creado para poder hacer una comparación entre los atributos 'name' de dos futuras instancias (droid1 y droid2).

```
>>> class Droid:
...     def __init__(self, name, serial_number):
...         self.serial_number = serial_number
...         self.name = name
...
...     def __eq__(self, droid):
...         return self.name == droid.name
...

>>> droid1 = Droid('C-3PO', 43974973242)
>>> droid2 = Droid('C-3PO', 85094905984)

>>> droid1 == droid2 # llamada implícita a __eq__
True

>>> droid1.__eq__(droid2)
True
```

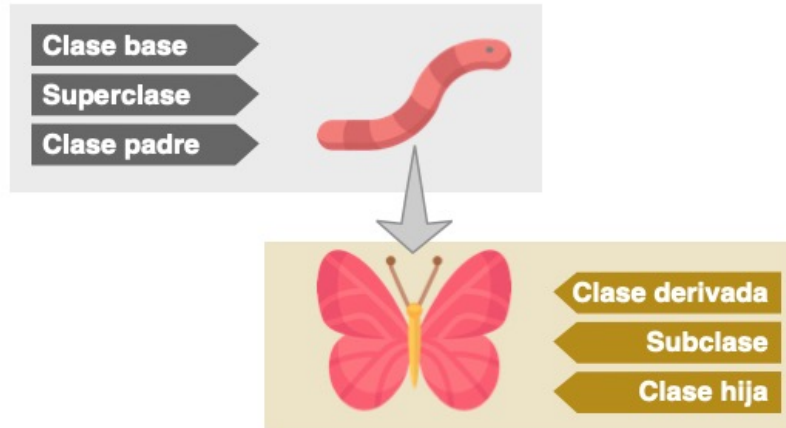


Herencia

6



La herencia es una característica consistente en seleccionar una clase ya existente y crear una nueva a partir de esta, pero con la capacidad de añadir o modificar algunos aspectos. Está considerado como una buena práctica porque se reutiliza código y se realizan generalizaciones.





Una vez hemos definido una clase desde la que heredar (superclase o clase base) simplemente tenemos que construir la nueva clase (clase derivada o subclase) haciendo referencia entre paréntesis a la superclase.

En la imagen de la derecha se aprecia como la definición de la clase 'ProtocolDroid' incluye entre sus parámetros a la clase 'Droid', de la que va a heredar sus métodos (y atributos si los tuviese).

```
>>> class Droid:
...     def switch_on(self):
...         print("Hi! I'm a droid. Can I help you?")
...
...     def switch_off(self):
...         print("Bye! I'm going to sleep")
...

>>> class ProtocolDroid(Droid):
...     pass
...

>>> r2d2 = Droid()
>>> c3po = ProtocolDroid()

>>> r2d2.switch_on()
Hi! I'm a droid. Can I help you?

>>> c3po.switch_on() # método heredado de Droid
Hi! I'm a droid. Can I help you?

>>> r2d2.switch_off()
Bye! I'm going to sleep
```




De nuevo vemos un ejemplo muy parecido al anterior en la imagen de la derecha. Tenemos una clase 'Droid' con dos métodos propios denominados 'switch_on()' y 'switch_off()'.

De igual forma se declara la clase 'ProtocolDroid' para heredar los métodos de la anterior. Sin embargo, esta vez se está declarando de nuevo el método 'switch_on()' (se sobrescribe dicho método). Éste tiene un comportamiento distinto en la clase heredera, con lo que obtendremos diferentes resultados si lo llamamos desde una clase u otra.

```
>>> class Droid:
...     def switch_on(self):
...         print("Hi! I'm a droid. Can I help you?")
...
...     def switch_off(self):
...         print("Bye! I'm going to sleep")
...

>>> class ProtocolDroid(Droid):
...     def switch_on(self):
...         print("Hi! I'm a PROTOCOL droid. Can I help you?")
...

>>> r2d2 = Droid()
>>> c3po = ProtocolDroid()

>>> r2d2.switch_on()
Hi! I'm a droid. Can I help you?

>>> c3po.switch_on() # método heredado pero sobrescrito
Hi! I'm a PROTOCOL droid. Can I help you?
```



No sólo es posible modificar un método ya existente en una superclase desde una clase derivada, sino que también se puede crear un nuevo método totalmente independiente de la superclase. Vemos como se ha definido el método 'translate' para la clase 'ProtocolDroid'. Si realizamos una llamada desde la clase 'Droid' Python nos devolverá un error avisandonos de que no existe tal método para dicha clase.

```
>>> class Droid:
...     def switch_on(self):
...         print("Hi! I'm a droid. Can I help you?")
...
...     def switch_off(self):
...         print("Bye! I'm going to sleep")
...

>>> class ProtocolDroid(Droid):
...     def switch_on(self):
...         print("Hi! I'm a PROTOCOL droid. Can I help you?")
...
...     def translate(self, msg, from_language):
...         ''' Translate from language to Human understanding '''
...         print(f'{msg} means "ZASCA" in {from_language}')

>>> r2d2 = Droid()
>>> c3po = ProtocolDroid()

>>> c3po.translate('kiitos', 'Huttese') # idioma de Watoo
kiitos means "ZASCA" in Huttese

>>> r2d2.translate('kiitos', 'Huttese') # droide genérico no puede traducir
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'Droid' object has no attribute 'translate'
```



En programación orientada a objetos, `super()` es una herramienta poderosa que permite a las clases derivadas (subclases) llamar a métodos o acceder a atributos de sus clases base (superclases). Esta funcionalidad es especialmente útil cuando queremos extender o modificar el comportamiento heredado en una subclase sin duplicar el código de la superclase.

```
python

class Vehículo:
    def __init__(self, marca, modelo):
        self.marca = marca
        self.modelo = modelo

    def arrancar(self):
        print(f"{self.marca} {self.modelo} ha arrancado.")

class Coche(Vehículo):
    def __init__(self, marca, modelo, puertas):
        # Llamar al constructor de la clase base
        super().__init__(marca, modelo)
        self.puertas = puertas

    def arrancar(self):
        # Llamar al método arrancar de la clase base
        super().arrancar()
        print(f"El coche tiene {self.puertas} puertas.")

# Crear una instancia de Coche
mi_coche = Coche("Toyota", "Corolla", 4)

# Llamar al método arrancar
mi_coche.arrancar()
```

Módulos y paquetes

7



Un módulo es un fichero de texto que contienen código escrito en Python, creado para contener unidades de código de interés que favorezcan la reutilización y eviten la repetición del mismo. Es por ello que es recomendable, cuanto más grande sea el proyecto, utilizar uno o varios módulos para albergar nuestro código de valor.

Estos módulos contienen la extensión `‘.py’`, y pueden formar parte de estructuras más grandes denominadas paquetes. Por definición un paquete es una carpeta que contiene ficheros `‘.py’`, y que además contiene un archivo de inicio denominado `‘__init__.py’`. No es necesario que este fichero tenga contenido o instrucciones de ningún tipo.

```
└─ paquete
   │
   └─ __init__.py
   │
   └─ modulo1.py
   │
   └─ modulo2.py
   │
   └─ modulo3.py
```

```
└─ .
   │
   └─ modulo1.py
   │
   └─ paquete
      │
      └─ __init__.py
      │
      └─ modulo1.py
      │
      └─ subpaquete
         │
         └─ __init__.py
         │
         └─ modulo1.py
         │
         └─ modulo2.py
```



Para sacar provecho de un módulo es necesario importarlo antes mediante la instrucción 'import' seguida del nombre del paquete que se quiere importar. También es posible acceder a un módulo dentro de uno o varios paquetes, evitando así cargar todo el módulo si no nos es de interés. Esto se realiza mediante la utilización del carácter '.', intercalando este con las diferentes subsecciones (namespaces) del módulo.

```
import modulo           # importar un módulo que no pertenece a un paquete
import paquete.modulo1  # importar un módulo que está dentro de un paquete
import paquete.subpaquete.modulo1
```

Por último, se puede abreviar un namespace importado mediante un alias con la construcción 'as'. Esto facilita el uso del módulo a lo largo del código si este es muy requerido.

```
import modulo as m
import paquete.modulo1 as pm
import paquete.subpaquete.modulo1 as psm
```

¡Gracias!